

Git y Gitflow / Introducción

FUNDAMENTOS

¿Qué es Git y para qué sirve?

Git es un sistema de control de versiones: registra la evolución de los archivos de un proyecto para que podamos entender los cambios, recuperar estados anteriores y trabajar en equipo.

i La idea antes que los comandos

Git no guarda simplemente "copias". Construye una historia formada por versiones relacionadas. Cada versión explica **qué se cambió, quién lo hizo y por qué**.

El problema que resuelve

Un proyecto cambia continuamente. Modificamos código, documentación, archivos de configuración, scripts de despliegue y recursos. Sin un sistema de control de versiones aparecen carpetas como estas:

Una historia difícil de mantener

```
proyecto/  
proyecto-copia/  
proyecto-final/  
proyecto-final-bueno/  
proyecto-final-definitivo/
```

Las copias permiten recuperar algún archivo, pero no responden bien a preguntas

Control de versiones con Git



- ¿Quién hizo el cambio y con qué intención?
- ¿Cómo combinamos el trabajo de varias personas?

Ventajas de utilizar Git



Recuperar

Podemos volver a una versión conocida o recuperar un archivo concreto.



Comparar

Git muestra qué líneas se añadieron, modificaron o eliminaron.



Trabajar en paralelo

Las ramas permiten desarrollar tareas aisladas y unir las más tarde.



Mantener trazabilidad

El historial relaciona cada cambio con un autor, una fecha y un mensaje.

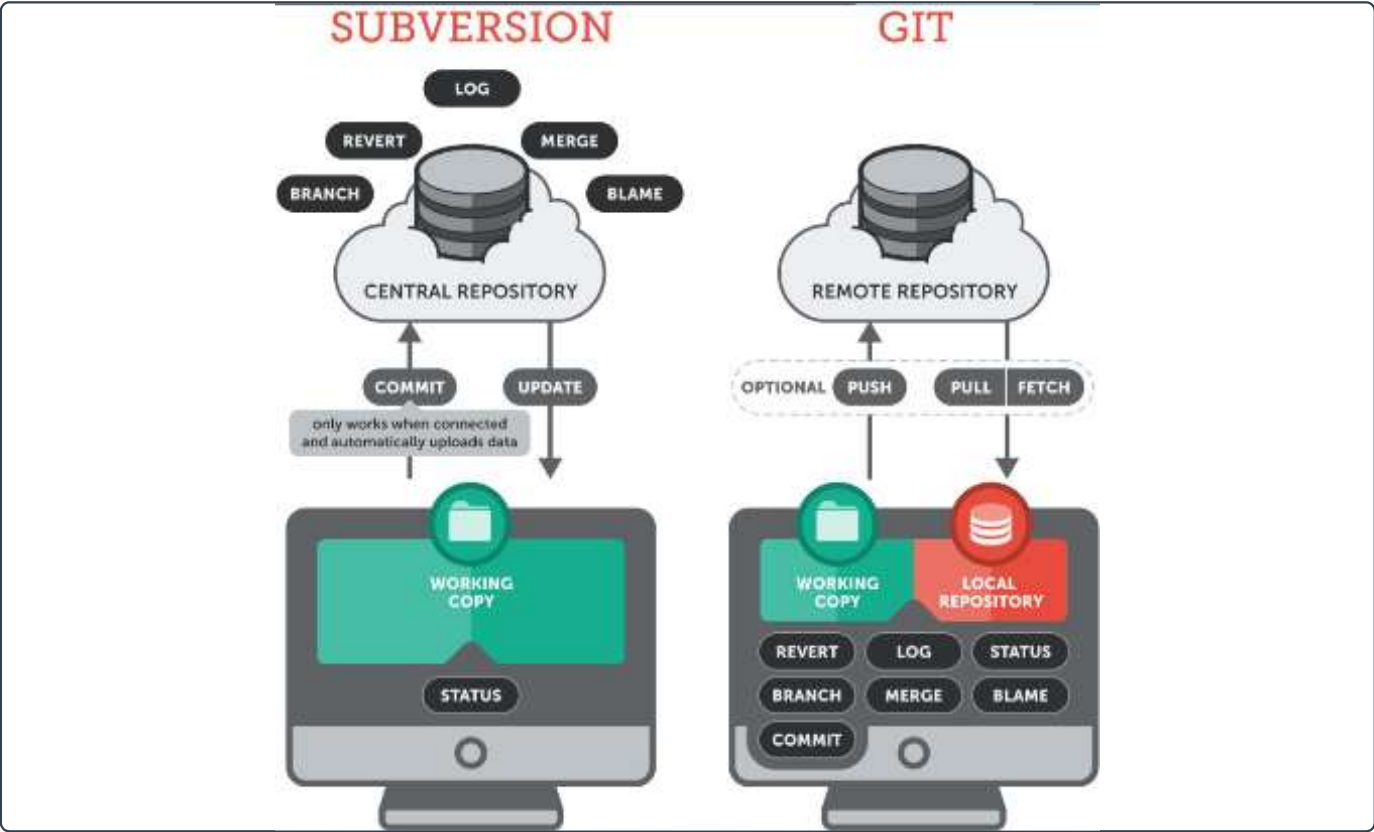
En despliegue no versionaremos únicamente código. Git también resulta útil para Dockerfile, compose.yaml, documentación Markdown, configuraciones de servidores, scripts y archivos de integración continua.

Control de versiones con Git



Los sistemas de control de versiones pueden organizar su historial de distintas formas.

Sistema centralizado	Sistema distribuido
El historial principal reside en un servidor.	Cada clon contiene el proyecto y su historial.
Los usuarios dependen más del servidor para consultar y registrar versiones.	Se pueden crear commits, ramas y comparaciones sin conexión.
El servidor es un punto central de administración y también un punto crítico.	Los repositorios intercambian cambios cuando se decide sincronizarlos.
Ejemplos: CVS y Subversion.	Ejemplo principal: Git.



En un sistema distribuido cada desarrollador conserva un repositorio local completo.

✓ Consecuencia práctica

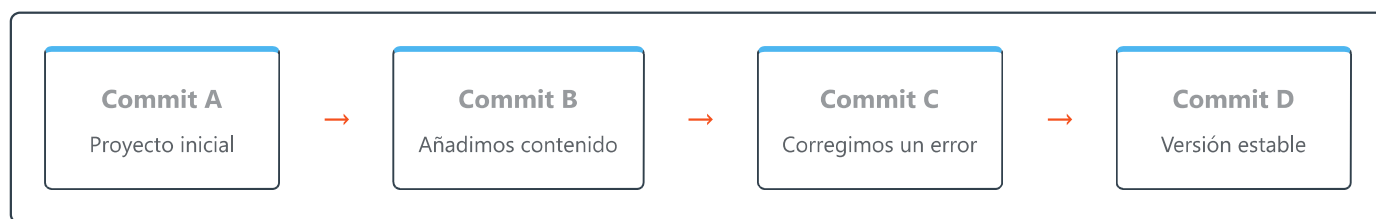


commits, revisar el historial o trabajar con ramas sin estar
e el equipo utilice GitHub como punto de encuentro, Git sigue



Git guarda una historia de commits

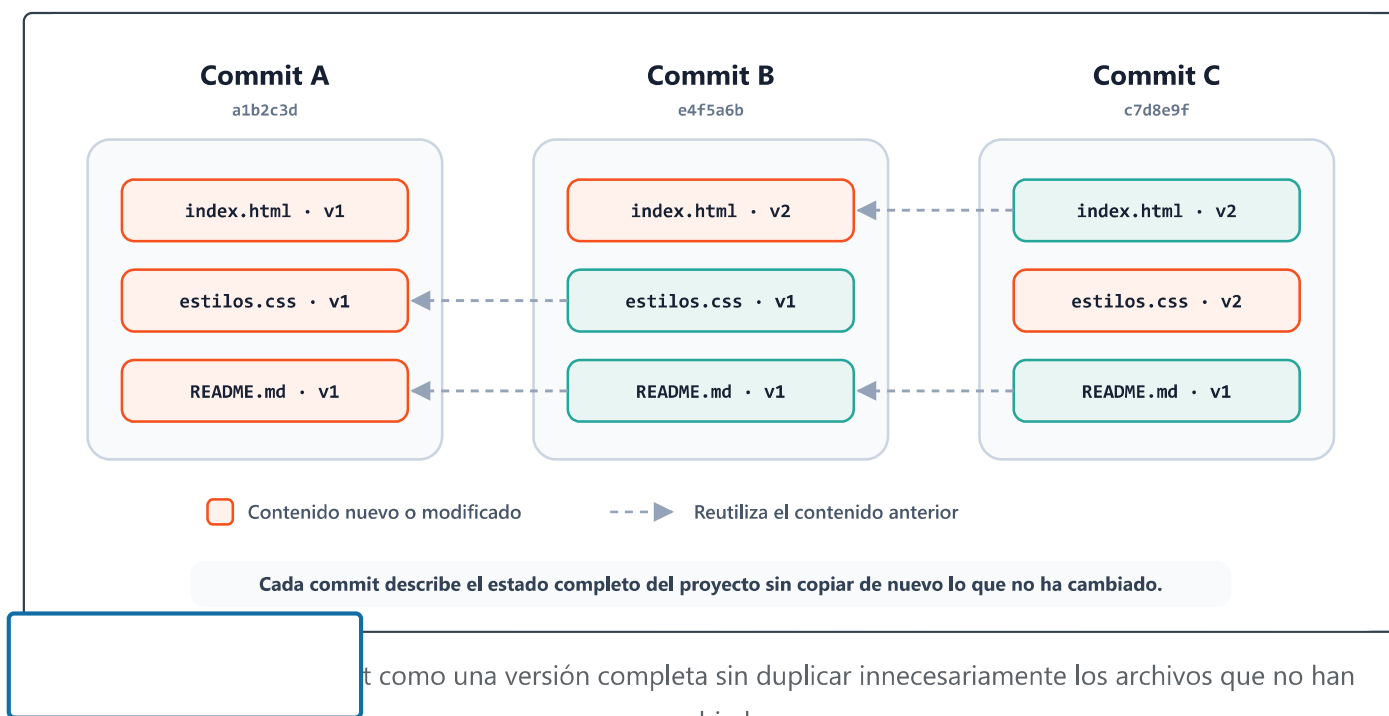
Git fue diseñado para manejar proyectos grandes y trabajo distribuido, pero su modelo también resulta útil en un proyecto individual. Cada versión importante se registra mediante un **commit**.



Un commit puede entenderse como una fotografía lógica del proyecto. Contiene un identificador, un autor, una fecha, un mensaje y una referencia al commit anterior.

Una instantánea, no una carpeta duplicada

Cada commit representa el estado completo del proyecto en ese momento. Cuando un archivo cambia, Git guarda su nuevo contenido; cuando no cambia, reutiliza el objeto que ya tenía almacenado.



Control de versiones con Git



Aunque suelen aparecer juntos, no son lo mismo:

Git

Es la herramienta de control de versiones. Funciona en nuestro equipo y crea repositorios, commits, ramas, comparaciones y fusiones.

GitHub y GitLab

Son plataformas que alojan repositorios Git y añaden permisos, incidencias, revisiones, pull requests y automatización.

i Git no necesita GitHub

Podemos utilizar Git durante todo el desarrollo sin publicar el repositorio. Más adelante veremos cómo conectar un repositorio local con GitHub.

SIGUIENTE

Cómo funciona Git →